

Hydra Wall Street Library - Requirements Documentation

Overview

This document defines the **functional and non-functional requirements** for the **Hydra Wall Street Library**. It translates the project goals into concrete, traceable specifications covering SDKs, data adapters, user interfaces, and telemetry.

The document also defines success metrics, delivery timelines, and operational requirements, ensuring feasibility, measurability, and alignment with Catalyst Milestone expectations.

1. Project Goals & User Outcomes

1.1 Project Goals

- Enable developers to experiment with Hydra in real-world finance-style scenarios
- Demonstrate deterministic, high-performance execution using Hydra Heads
- Provide reusable open source libraries, SDKs, and tooling
- Reduce complexity and time-to-market for Hydra-based financial applications

1.2 Target Users

- Cardano developers
- Quant and algorithmic trading researchers
- Blockchain infrastructure engineers
- Educators and researchers studying market microstructure

All requirements below are explicitly traceable to these goals and user groups.

2. Functional Requirements

2.1 SDK Requirements

The system SHALL provide developer SDKs that:

- Allow joining or spawning Hydra Heads
- Support submission, cancellation, and modification of orders
- Expose deterministic event streams (fills, rejects, snapshots)
- Provide replay and backtesting helpers
- Abstract Hydra WS and HTTP interfaces behind stable APIs

Supported SDKs:

- TypeScript (web and Node.js)
 - Python (quant and research workflows)
-

2.2 Data Adapter Requirements

The system SHALL include data adapters that:

- Ingest seeded synthetic market data deterministically
- Support optional delayed real-world datasets
- Normalize data into a common internal schema
- Allow pluggable asset classes (equities, FX, crypto)

Data adapters MUST:

- Be replayable from a known seed or timestamp
 - Produce identical outputs given identical inputs
-

2.3 Order Book & Execution Requirements

The execution engine SHALL:

- Implement price-time priority matching
- Support limit, IOC, cancel, and partial fill orders
- Emit deterministic execution events

- Produce consistent P&L and position tracking

All execution logic MUST be deterministic and snapshot-compatible.

2.4 User Interface Requirements

The UI SHALL provide:

- Order entry and cancellation controls
- Level 2 order book depth visualization
- Trade tape and execution history
- Real-time P&L tracking
- Latency and throughput indicators
- Hydra Head lifecycle and connection status

The UI is intended for:

- Developer debugging
 - Demonstration and education
-

2.5 Telemetry & Observability Requirements

The system SHALL expose telemetry for:

- Order latency (submission → acknowledgment)
- Throughput (orders/sec, events/sec)
- Snapshot frequency and processing time
- WS reconnect counts and durations
- Replay success/failure indicators

Telemetry MUST be accessible via:

- SDK hooks
 - Metrics endpoints
 - Logs suitable for CI and benchmarking
-

3. Non-Functional Requirements

3.1 Performance Requirements

Metric	Target
Order acknowledgment latency	< 50 ms
WS event latency	< 100 ms
Snapshot recovery time	< 200 ms
Sustained throughput	No SDK bottleneck

3.2 Reliability Requirements

The system MUST:

- Recover from WS disconnects using snapshots
 - Detect and prevent state divergence
 - Resume cleanly after process restarts
 - Fail fast if determinism guarantees are violated
-

3.3 Security Requirements

The system MUST:

- Avoid custody of real assets
 - Prevent replay attacks via sequencing
 - Validate all inbound messages
 - Isolate untrusted data feeds
-

3.4 Compatibility Requirements

- Compatible with latest stable hydra-node
 - Runnable via containers for local and CI usage
 - SDKs usable on clean developer machines
-

4. Operational Requirements

4.1 Sync Handling

- Snapshot-based recovery is mandatory
- Events without a snapshot baseline MUST be rejected

4.2 Failover Expectations

- Clients MUST reconnect automatically
- State MUST resynchronize deterministically

4.3 Data Freshness Expectations

Data Type	Expectation
Synthetic data	Deterministic, seed-based
Delayed real data	Clearly labeled, non-authoritative
Execution state	Snapshot-authoritative

5. Success Metrics

Success will be measured by:

- Ability to run a fully deterministic end-to-end simulation
- Reproducible CI runs with identical results
- Measured latency and throughput within targets
- Successful on-chain anchoring of state hashes
- External developers running examples without custom setup

6. Delivery Timeline

Month	Deliverables
Month 1	Launch docs, landscape review, technical assessment
Month 2	Architecture blueprint, APIs, risk evaluation

Month 3	Alpha implementation, SDKs, UI, CI
Month 4	Public release, hardening, documentation

All milestones are scoped to be feasible within the allocated budget and team size.

7. Risk Mitigation Summary

Risk	Mitigation
Sync loss	Snapshot-driven recovery
Data gaps	Deterministic sequencing
Replay errors	Hash-based verification
Performance variance	Optimized WS pipeline
Developer friction	Examples and templates